

Lépésről lépésre: KönyvBolt gyakorló feladat

Sorrend: konzolos, grafikus, backend, CSS, React. Minden lépésnél: mit csinálj, a kód, és egy sor arról, miért.

1. KONZOLOS C# (15 pont)

1. lépés: a projekt (1 pont)

Visual Studio: **Create a new project, Console App**, neve pontosan **Konyvesbolt**. A **konyvek.csv**-t másold a projekt mappájába, a fájl tulajdonságainál: **Copy to Output Directory = Copy if newer**. Miért: ha a csv nem kerül a kimeneti mappába, a program nem találja a fájlt.

2. lépés: a Konyv osztály (4 pont)

Jobb gomb a projekten, **Add, Class, Konyv.cs**:

```
internal class Konyv
{
    public string Cim { get; set; }
    public string Szerzo { get; set; }
    public string Kiado { get; set; }
    public string Kategoria { get; set; }
    public int Ev { get; set; }
    public int Ar { get; set; }
    public int Oldalszam { get; set; }

    public Konyv(string sor)
    {
        string[] m = sor.Split(';');
        Cim = m[0];
        Szerzo = m[1];
        Kiado = m[2];
        Kategoria = m[3];
        Ev = int.Parse(m[4]);
        Ar = int.Parse(m[5]);
        Oldalszam = int.Parse(m[6]);
    }
}
```

Miért: a tulajdonságok a csv fejlécének sorrendjét követik, a számokat **int.Parse** alakítja át.

3. lépés: beolvasás listába (2 pont)

Program.cs:

```
using System.Globalization;
using System.Text;

List<Konyv> konyvek = new List<Konyv>();
foreach (string sor in File.ReadAllLines("konyvek.csv", Encoding.UTF8).Skip(1))
{
    konyvek.Add(new Konyv(sor));
}
```

Miért: a **.Skip(1)** kihagyja a fejléct, mert az nem alakítható számmá.

4. lépés: darabszám (2 pont)

```
Console.WriteLine("4. feladat: A könyvek száma: {0} db", konyvek.Count);
```

5. lépés: Alexandra, legfeljebb 3000 Ft (2 pont)

```
Console.WriteLine("5. feladat: Alexandra kiadó, legfeljebb 3000 Ft-os könyvek:");
foreach (Konyv k in konyvek.Where(x => x.Kiado == "Alexandra" && x.Ar <= 3000))
{
    Console.WriteLine(k.Cim);
}
```

Miért: a **Where** a feltételnek megfelelőket adja vissza, a **&&** mindkét feltételt kéri.

6. lépés: az Akcios módszer (2 pont)

A **Konyv** osztályba, a konstruktor alá:

```
public bool Akcios()
{
    return Kategoria == "Gyerek" && Ar < 2000;
}
```

Nem kell meghívni.

7. lépés: átlagár kiadó szerint (2 pont)

```
Console.Write("7. feladat: Kérem a kiadó nevét: ");
string kiado = Console.ReadLine();
var talalat = konyvek.Where(x => x.Kiado.ToLower() == kiado.ToLower()).ToList();
if (talalat.Count > 0)
{
    double atlag = talalat.Average(x => x.Ar);
    Console.WriteLine("A kiadó könyveinek átlagára: " + atlag.ToString("0.0", new CultureInfo("hu-HU")));
}
else
{
    Console.WriteLine("Nem található ilyen kiadó.");
}
```

Ellenőrzés: 60 db, a lista 6 cím, Magvető átlaga 3202,5. A tizedesvessző a hu-HU kultúra miatt van.

2. GRAFIKUS C# (10 pont)

1. lépés: az adatbázis (1 pont)

XAMPP: MySQL és Apache bekapcsol, böngészőben `localhost/phpmyadmin`, **Adatbázisok:** név `konyvesbolt`, rendezés `utf8mb4_hungarian_ci`, Létrehozás. SQL-lel:

```
CREATE DATABASE konyvesbolt CHARACTER SET utf8mb4 COLLATE utf8mb4_hungarian_ci;
```

2. lépés: az SQL importálása

A `konyvesbolt` adatbázist kijelölve: **Importálás**, `konyvesbolt.sql`, Végrehajtás. Két tábla jön létre: `kategoriak` és `konyvek` (40 sor).

3. lépés: a WPF projekt (1 pont)

Új **WPF Application**, neve pontosan `KonyvesboltGUI`. Jobb gomb a projekten, **Add, Existing Item**, `Konyv.cs` (a namespace `KonyvesboltGUI` legyen). **Manage NuGet Packages:** telepítsd a `MySQL.Data` csomagot.

4. lépés: a felület (3 pont)

`MainWindow.xaml`, a `<Grid>` belseje (az ablak `Title`-je: `KonyvesboltGUI`):

```
<Grid>
    <DataGrid x:Name="dgKonyvek" AutoGenerateColumns="False" IsReadOnly="True"
        SelectionMode="Single" HorizontalAlignment="Left" Width="500" Margin="10">
        <DataGrid.Columns>
            <DataGridTextColumn Header="Cím" Binding="{Binding Cim}" Width="*" />
            <DataGridTextColumn Header="Szerző" Binding="{Binding Szerzo}" Width="*" />
            <DataGridTextColumn Header="Ár" Binding="{Binding Ar}" Width="Auto" />
        </DataGrid.Columns>
    </DataGrid>

    <StackPanel HorizontalAlignment="Left" Margin="520,10,0,0" Width="250">
        <Button x:Name="btnKategoria" Content="4. feladat" Click="btnKategoria_Click" Margin="0,0,0,5" />
        <Label x:Name="lblKategoria" Content="Kategória: " Margin="0,0,0,20" />
        <Button x:Name="btnDarab" Content="5. feladat" Click="btnDarab_Click" Margin="0,0,0,5" />
        <Label x:Name="lblDarab" Content="" />
    </StackPanel>
</Grid>
```

Miért: a `SelectionMode="Single"` a feladat kérése, a `Binding` nevek a `Konyv.cs` tulajdonságai.

5. lépés: az adatok betöltése és a kijelölés (2 pont)

`MainWindow.xaml.cs`:

```
using MySQL.Data.MySQLClient;
using System.Collections.Generic;
using System.Linq;
using System.Windows;

namespace KonyvesboltGUI
{
    public partial class MainWindow : Window
    {
        List<Konyv> konyvek = new List<Konyv>();
        MySqlConnection connection = new MySqlConnection("server=localhost;database=konyvesbolt;uid=root;password='';");

        public MainWindow()
        {
            InitializeComponent();

            connection.Open();
            MySqlCommand cmd = new MySqlCommand("SELECT * FROM konyvek", connection);
            using (MySqlDataReader reader = cmd.ExecuteReader())
            {
                while (reader.Read())
                {
                    konyvek.Add(new Konyv(reader));
                }
            }
            connection.Close();

            dgKonyvek.ItemsSource = konyvek;
            dgKonyvek.SelectedIndex = 0;
        }
    }
}
```

6. lépés: a 4. feladat gombja (2 pont)

Ugyanabban az osztályban:

```
private void btnKategoria_Click(object sender, RoutedEventArgs e)
```

```

{
    Konyv kijelolt = dgKonyvek.SelectedItem as Konyv;

    connection.Open();
    MySqlCommand cmd = new MySqlCommand(
        "SELECT katNev FROM kategoriak WHERE katId = " + kijelolt.KategoriaId, connection);
    string nev = cmd.ExecuteScalar().ToString();
    connection.Close();

    lblKategoria.Content = "Kategória: " + nev;
}

```

7. lépés: az 5. feladat gombja (2 pont)

```

private void btnDarab_Click(object sender, RoutedEventArgs e)
{
    int db = konyvek.Count(x => x.Ar >= 3000);
    lblDarab.Content = "Legalább 3000 Ft-os könyvek: " + db;
}
}

```

Ellenőrzés: induláskor az 1. sor kijelölt, a 4. gomb **Kategória: Gyerek**-et ír, az 5. gomb **23**-at.

3. BACKEND (15 pont)

1. lépés: környezet és admin (1 pont)

A `backend` mappában:

```
python -m venv .
Scripts\activate
pip install -r requirements.txt
python manage.py createsuperuser
```

Felhasználónév `boltadmin`, jelszó `Bolt2026` (a jelszót kétszer kéri, a "túl egyszerű" figyelmeztetésre `y`).

2. lépés: a Valasz modell (7 pont)

`konyvesbolt/models.py` aljára:

```
class Valasz(models.Model):
    velemeny = models.ForeignKey(Velemeny, on_delete=models.CASCADE)
    nev = models.CharField(max_length=100)
    valasz_szoveg = models.CharField(max_length=200)
    hasznos_szazalek = models.IntegerField()
    valaszolt_ekkor = models.DateTimeField()
```

Miért: `on_delete=models.CASCADE` a kaszkádolt törlés, a `CharField`-nél a `max_length` kötelező.

3. lépés: migráció (1 pont)

```
python manage.py makemigrations
python manage.py migrate
```

4. lépés: admin felület (1 pont)

`konyvesbolt/admin.py`:

```
from .models import Velemeny, Valasz
```

```
admin.site.register(Velemeny)
admin.site.register(Valasz)
```

5. lépés: rekord felvétele (1 pont)

`python manage.py runserver`, majd `http://127.0.0.1:8000/admin`, belépés, **Valasz**, Add: tetszőleges adatokkal.

6. lépés: a név megjelenítése (1 pont)

A `Valasz` osztályba:

```
def __str__(self):
    return self.nev
```

7. lépés: serializer (1 pont)

`serializers.py` (az importot is bővítsd: `from .models import Velemeny, Valasz`):

```
class ValaszSerializer(serializers.ModelSerializer):
    class Meta:
        model = Valasz
        fields = "__all__"
        depth = 1
```

8. lépés: view (1 pont)

`views.py` (importok: `Valasz, ValaszSerializer`):

```
@api_view(["GET"])
def getValaszok(request):
    osszes = Valasz.objects.all()
    serialized = ValaszSerializer(osszes, many=True)
    return Response(serialized.data)
```

9. lépés: URL (1 pont)

`konyvesbolt/urls.py`, az `urlpatterns` listába:

```
path('valaszok/', views.getValaszok, name="getValaszok"),
```

Ellenőrzés: `http://127.0.0.1:8000/api/valaszok/`, a `velemeny` beágyazott objektumként látszik (ez a `depth = 1`).

4. CSS (10 pont): minden az exam.css-be

1-2. lépés: kép és útvonal (2 pont)

```
img {
  border-radius: 5px;
}

.utvonal {
  color: gray;
}

.utvonal span {
  color: black;
}
```

3. lépés: a gomb hover (2 pont)

Jobb gomb a kék "Kosárba" gombon, **Inspect**: az osztálya `kosarGomb`.

```
.kosarGomb:hover {
  background-color: var(--color-kek-arnyalat);
}
```

4. lépés: Flexbox (2 pont)

```
.menusor {
  display: flex;
  justify-content: space-between;
}
```

5. lépés: Grid (2 pont)

```
.adatok {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 20px;
}
```

6. lépés: reszponzív (2 pont)

A fájl **végére**:

```
@media (max-width: 800px) {
  .velemenyekFo {
    grid-template-columns: 1fr;
  }
}
```

Ellenőrzés: szűkítsd az ablakot 800 px alá, a vélemények szakasza egy oszlopra vált.

5. REACT (15 pont)

1-2. lépés: a komponens és exportja (2 pont)

`src/components/VelemenyKartya.jsx`:

```
function VelemenyKartya() {
  return <div></div>
}
```

`export default VelemenyKartya`

3-4. lépés: import és state a Velemenyek-ben (3 pont)

`Velemenyek.jsx`:

```
import VelemenyKartya from './VelemenyKartya.jsx'
import { useState, useEffect } from 'react'
```

```
function Velemenyek() {
  const [apiVelemenyek, setApiVelemenyek] = useState([])
```

5-6. lépés: kirajzolás és props (3 pont)

A `return` a "velemenyek" osztályú div-ben:

```
return (
  <div className="velemenyek">
    {apiVelemenyek.map(v => (
      <VelemenyKartya
        key={v.id}
```

```

        nev={v.nev}
        szoveg={v.szoveg}
        tetszes={v.tetszes}
        datum={v.datum}
      />
    ))}
  </div>
)
}

```

export default Velemenyek

7. lépés: lekérés (2 pont)

A state alá, a **return** elé:

```

useEffect(() => {
  fetch('/velemenyekek.json')
    .then(response => response.json())
    .then(data => setApiVelemenyek(data))
}, [])

```

Backendről: `fetch('http://127.0.0.1:8000/api/velemenyekek/')`. A `[]` miatt csak egyszer fut le.

8-10. lépés: a VelemenyKartya kidolgozása (5 pont)

```
import { useState } from 'react'
```

```

function VelemenyKartya(props) {
  const [tetszes, setTetszes] = useState(props.tetszes)

  return (
    <div className="velemenyekek">
      <header>
        <strong>{props.nev}</strong>
        <i className="fa-regular fa-flag"></i>
      </header>
      <div className="datumDoboz">{(new Date(props.datum)).toLocaleDateString()}</div>
      <p>{props.szoveg}</p>
      <div className="tetszesDoboz">
        <button className="tetszesGomb" onClick={() => setTetszes(tetszes + 1)}>
          <i className="fa-regular fa-thumbs-up"></i>
        </button> Tetszik ({tetszes})
      </div>
    </div>
  )
}

```

export default VelemenyKartya

Miért: a név, dátum és szöveg a **props**-ból jön, a számot a **state**-ből írjuk ki, az **onClick**-ben nyílfüggvény kell. Ellenőrzés: 5 vélemény jelenik meg, a gombra kattintva csak annál nő a szám.